

2. 数据表示与编码

2026 春

杨莉萍
信息学院大数据系

本章主要内容

- 非数值数据表示法
- 数值数据表示法
- 数据信息的校验。

非数值数据表示法 (※)

主要内容：

- (西文)字符表示法
- 汉字表示法。

128 Standard ASCII codes

- 52个 字母

 - a-z, A-Z

- 10个 数字

 - '0' -- '9'

- 33 符号

 - ! @ # \$ % ^ & * () 等

- 33 个Control characters (第0 ~ 31号, 第127号) 通信控制码(非打印)
参见P40表 .

 - <CR> <ESC> 等。

- 基本ASCII码为7bit, 在内存占1字节, 最高位为0 .
- **字符串的存放方法**: 通常它们占用主存中连续的多个字节, 每个字节存放一个西文字符的ASCII码值。

汉字表示法

- 1字节8bit数据仅能表示256个字符, 而常用汉字达6000多个, 故1字节无法表示汉字; 因此**汉字编码采用双字节**.

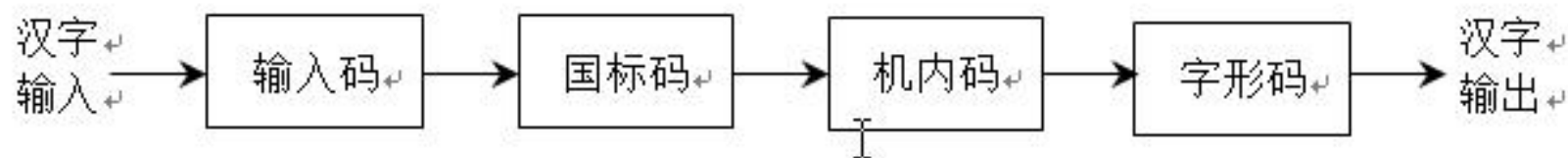
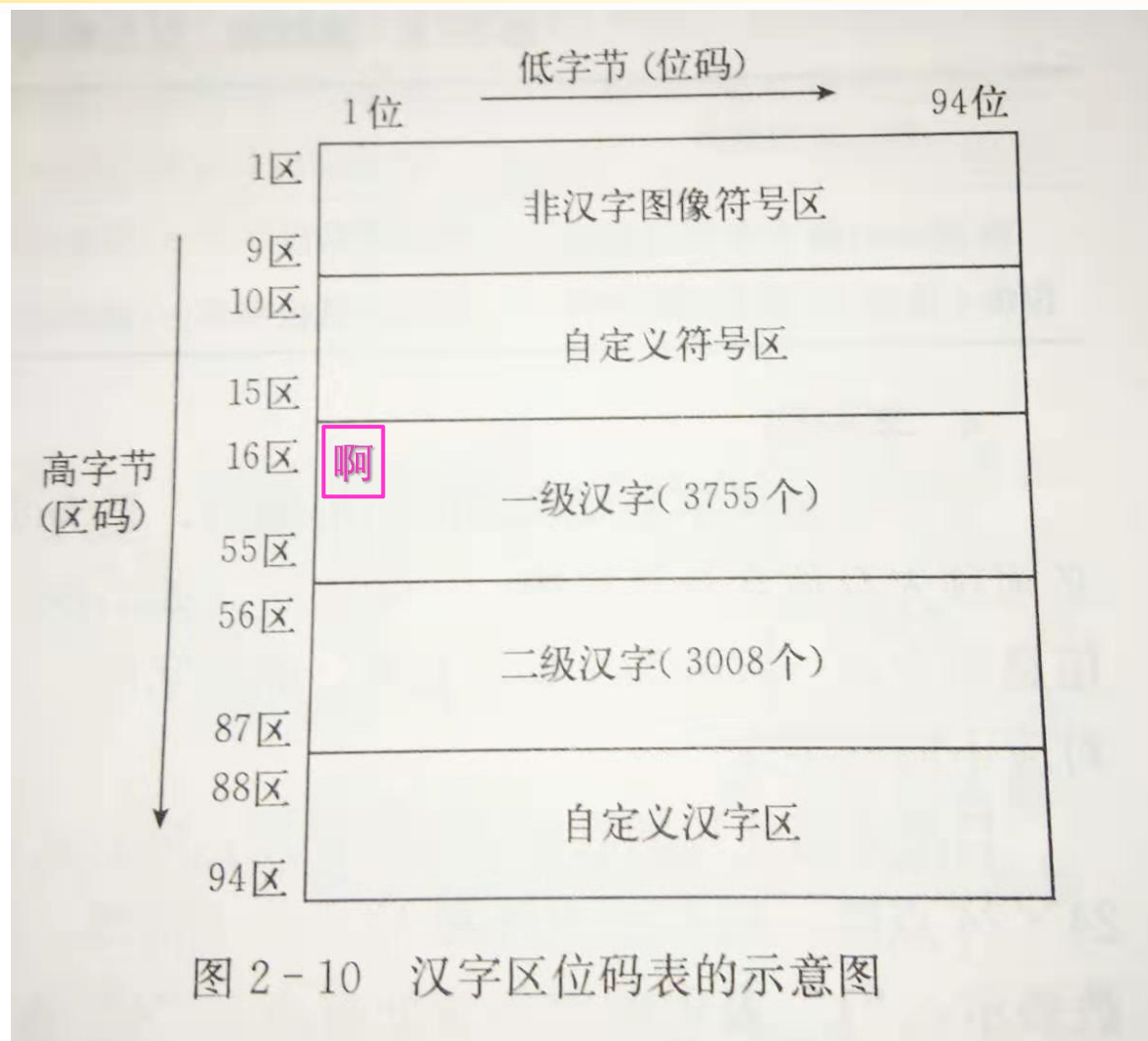


图 2-9 汉字信息处理过程

- **GB2312国家标准:**
 - 16bit表示, 又称为**国标码** 或国际交换码, 共表示7445个中文字符
 - ◆ 常用汉字6763个
 - ◆ 非汉字全角图形字符682个 (标点符、运算符等).
- GB2312国标码的编码基于**区位码**。

区位码

- 7445个字符 排成94*94(=8836) 矩阵, 采用**区位码**(行号--区码, 列号--位码), 表示成2个十进制数; 这2个十进制数分别转换为二进制, 共2字节, 编码方式见右图 .
- 例: 第一个汉字 “啊” 出现在第16区第1位上, 其区位码为16 01, 用2字节的二进制数表示为00010000 00000001
- 区位码无法用于汉字通信, 因它可能与**通信控制码**(ASCII码0—31)有冲突。因此将每个汉字的区码和位码分别加上32 (0x20), 以避免与通信控制码发生冲突。经过这样的处理而得到的编码即 **国标码**。



汉字 机内码

- 是计算机内存储和处理汉字时使用的编码.
 - 因为需与ASCII码严格区分：所以, 汉字机内码(2字节)的每字节最高位为 1
 - 与区位码之间的简单对应：
汉字机内码 = 区位码 + 0x A0A0。对应转换原理见下页。

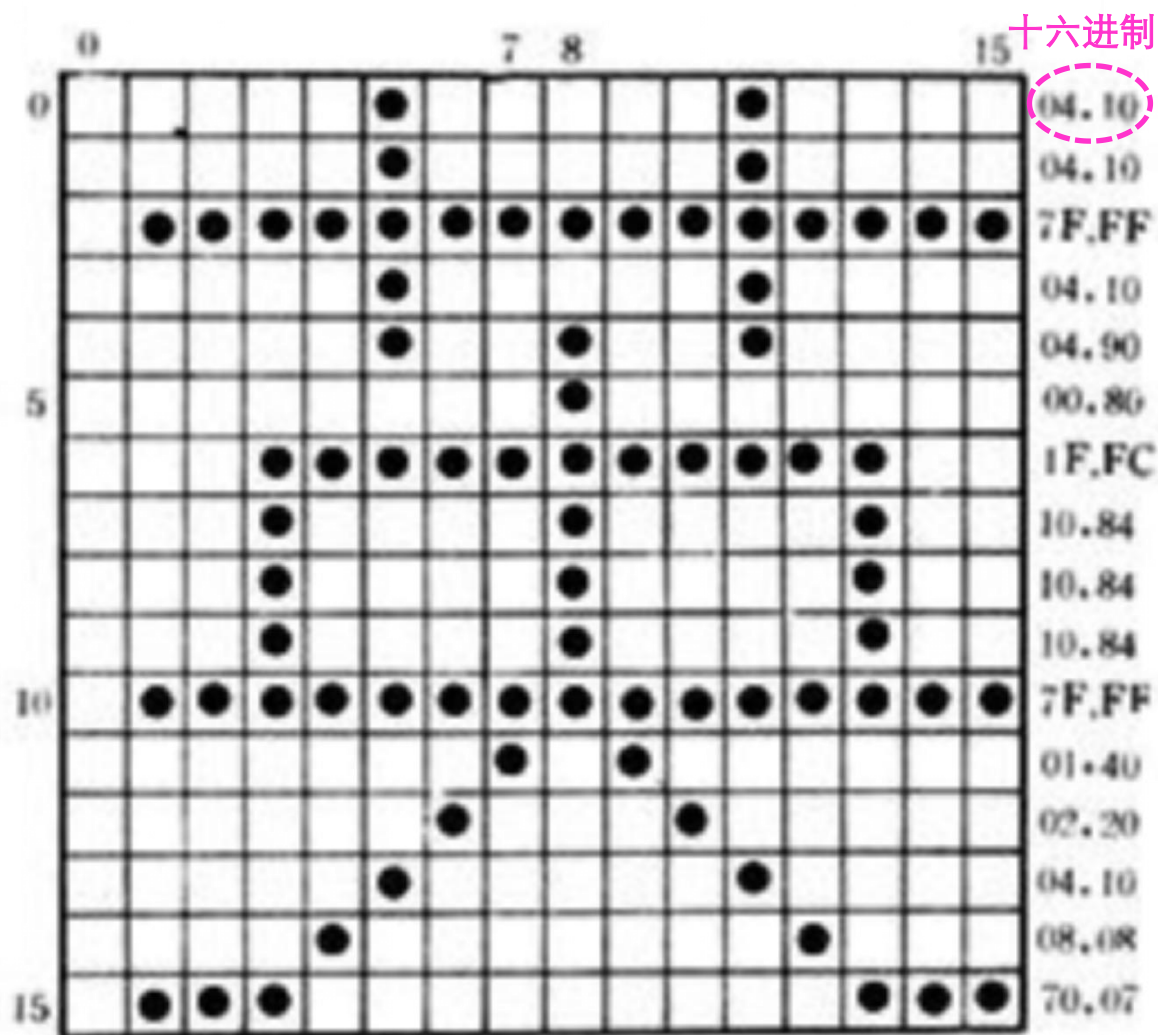
汉字区位码与机内码之间的转换

- GB2312字符集中每个汉字或符号用它所在的区号 连上 位号表示。每个汉字的区号和位号在计算机内部分别用1字节来表示。例, “学” 字的区号为49,位号为07, 它的区位码用2个字节的二进制数表示为: 00110001 00000111 。
- 区位码无法用于汉字通信, 因为它可能与通信使用的控制码(00H~1FH) (即 0~31) 发生冲突; 因此ISO2022规定每个汉字的区号和位号必须分别加上32 (即二进制数0010 0000) , 经过这样的处理而得的代码称为**国标交换码**, 简称 交换码; 因此, “学” 字的国标交换码计算为:
 - 0011 0001 0000 0111 区位码
 - + **0010** 0000 + **0010** 0000
 - -----
 - **0**101 0001 **0**010 0111 **交换码**
 - 用十六进制数表示即为5127H。

- 由于文本中通常混合使用汉字和西文字符，汉字信息如果不予以特别标识，就会与单字节西文字符的ASCII码混淆。此问题的解决方法之一是将一个汉字看成是两个扩展ASCII码，**使表示GB2312汉字的两个字节的最高位都为1**。这种高位为1的双字节汉字编码即为GB2312**汉字的机内码**，简称 **内码**。
- 因此，“学” (交换码 **0**1010001 **0**0100111) 的机内码：
 - 机内码 **1**1010001 **1**0100111
- 综上，(2字节) **机内码**
 - = 区位码的**区码**、**位码 分别** + **1010** 0000 (即0x **A0**)
 - = (2字节) **区位码** + 0x **A0 A0**。

字模码 (字形码)

- 字模码是用点阵表示的汉字字型代码，是汉字的输出形式。
- 字模点阵的信息量很大，因此 占存储空间很大。以16*16字形点阵为例(如图)，每个汉字要占用32字节；
- 所以字模点阵只能构成汉字库用来输出，不能用于机内存储。



主要内容：

- 计算机数值数据表示的特点
- 数制转换
- 机器码
- 数的定点、浮点表示。

不同进制编码特点

■ 十进制

- 有0~9共10种状态，状态过多。

■ 二进制

- 符号数最少，“0、1”物理上容易实现(数字电路)
- 可以表示任何对象(字符，数值，逻辑值)
- 运算规则简单:

◆ 二进制加法: $0+0=0$ $0+1=1+0=1$ $1+1=\underline{0}$

◆ 根据上述加运算规则, 用一个异或门(数电)即可完成加运算。



进制表示

**r进制数对应的十进制值——
位权展开和式：**

$$N = \sum_{i=-k}^m D_i * r^i$$

- N 代表十进制数值 r 是该数制的基数
- i 表示这些数码排列的位号 (该数码 后面 整数部分的位数, 小数部分取负值)
- D_i 是位号为 i 的位上的数码 (对应十进制的值)
- r^i 是位号为 i 的位上的 1 代表的值 (位权)
- $D_i * r^i$ 是第 i 位数码所代表的实际值, 表示从整数到小数部分所有位的值求累加和。

■ 任意进制转十进制 例:

■ $(10456.9)_{10} = 1 \times 10^4 + 0 \times 10^3 + 4 \times 10^2 + 5 \times 10^1 + 6 \times 10^0 + 9 \times 10^{-1} = 10456.9$

■ $(F96)_{16} = 15 \times 16^2 + 9 \times 16^1 + 6 \times 16^0 = \dots$

■ $(10010001.01)_2 = 1 \times 2^7 + 1 \times 2^4 + 1 \times 2^0 + 1 \times 2^{-2}$



主要内容:

- 二进制数 转 八进制
- 二进制数转十六进制
- 二进制数转十进制
- 十进制数转二进制。

二 到 八/十六进制 转换

■ 二进制转八进制

从小数点向左、右3位一分组

$$(10\ 011\ 100 . 01)_2$$

$$(010\ 011\ 100 . 01\underline{0})_2 = (234 . 2)_8 \text{ 对小数部分最低位, 必须加0 补齐这组!}$$

■ 二进制转十六进制

从小数点 向左、右 4位一分组

$$(1001\ 1100 . 01)_2$$

$$(1001\ 1100 . 01\underline{00})_2 = (9C . 4)_{16}$$

小数部分 不足位数要补零凑足 , 否则出错!

反之,八/十六进制如何转二进制? 例如 $(234 . 2)_8 = (?)_2$

十进制转二进制

■ 整数部分, 除2 取余

$$\begin{array}{r} 2 \overline{) 11 \text{ --- } 1} \text{ 低} \\ 2 \overline{) \quad 5 \text{ --- } 1} \text{ 写} \\ 2 \overline{) \quad 2 \text{ --- } 0} \text{ 结果} \\ 2 \overline{) \quad 1 \text{ --- } 1} \text{ 方向} \\ \quad \quad \quad 0 \text{ 高} \end{array}$$

除到商为0止: 1011

小数部分, 乘2 取整

乘积	乘积
整数部分	小数部分
结果 0.	0.625 * 2
高 1	0.25 * 2
0	0.5 * 2
低 1	0.0

直到乘积小数部分为0, 或求得位数满足要求为止: 0.101。

进制转换的简便运算方法

■ 引例: 十进制17/128 转换成二进制 ?

■ 对于大数的转换, 记住常用的2的幂:

$$2^5 = 32 \quad 2^6 = 64 \quad 2^7 = 128 \quad 2^8 = 256 \quad 2^9 = 512$$

$$2^{10} = 1024 \text{ (1K)} \quad 2^{11} = 2048 \quad 2^{12} = 4096$$

$$2^{13} = 8192 \quad 2^{15} = 32768$$

$$2^{16} = 65536$$

$$2^{20} = 1\text{M} \approx 10^6 \quad 2^{30} = 1\text{G}$$

$$\text{更大单位: } 2^{40} = 1\text{T} \quad 2^{50} = 1\text{P}。$$

几个简化运算的例子:

■ $130 = (\quad)_2 ?$

■ $= 128 + 2 = 2^7 + 2^1 = 10000010 \text{ B}$

■ $2010 = (\quad)_2 ?$

■ $= 2047 - 37 = 111 \ 1111 \ 1111 \underline{B} - (32 + 4 + 1)$

■ $= 11111 \ 1 \ 1111 \ 1 - 100101$

■ $= 11111 \ 011010$

■ $111111110111 \text{ B} = (\quad)_{10} ?$

■ $= 11111111 \ 1111 - 1000$

■ $= (2^{12} - 1) - 2^3$

■ $= 4096 - 9$

■ $= 4087$

■ $17/128 = (10001 / 10000000)_2 = 0.0010001 \text{ B}$ (小数点左移7位)。

作业 1

- **以下作业要求在本子上写出具体计算过程，回头QQ上拍照提交：**
- 1、将二进制数1111010 .00111101 B 转换为八进制数和十六进制数 .
- 2、用简便方法将十进制数13/128转换为二进制数。

■ 真值 (书写用)

□ 用+ - 符号表示数据的正负 的二进制数称为 **有符号数的真值** .

■ 机器不能识别书写格式(+ - 符号), 计算机内部如何表示负数?

■ 机器码 (机器内部使用)

□ 将符号和数值一起编码表示的二进制数称为 **机器码** .

■ 四种机器码:

□ 原码(Signed magnitude) 反码(One's complement)

□ 补码(Two's complement) 移码(Biased notation)。

1. 原码表示法

- 计算机如何表示数的正负？
- ——增加符号位：
- **最高位作为符号位，0表示正，1表示负，原码中数值位不变**

$$\text{纯整数}x: [X]_{\text{原}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^n - X & -2^n < X \leq 0 \end{cases}$$

例：

[1110]原 = 01110

[-1110]原 = 11110

$$\text{纯小数}x: [X]_{\text{原}} = \begin{cases} X & 0 \leq X < 1 \\ 1 - X & -1 < X \leq 0 \end{cases}$$

例：

[0.1101]原 = 0.1101

[- 0.1101]原 = 1.1101

原码的特性

- 直观易懂

- 第一位是符号位，其他是数值位

- 正零、负零两个零（**0**的表示不唯一）：

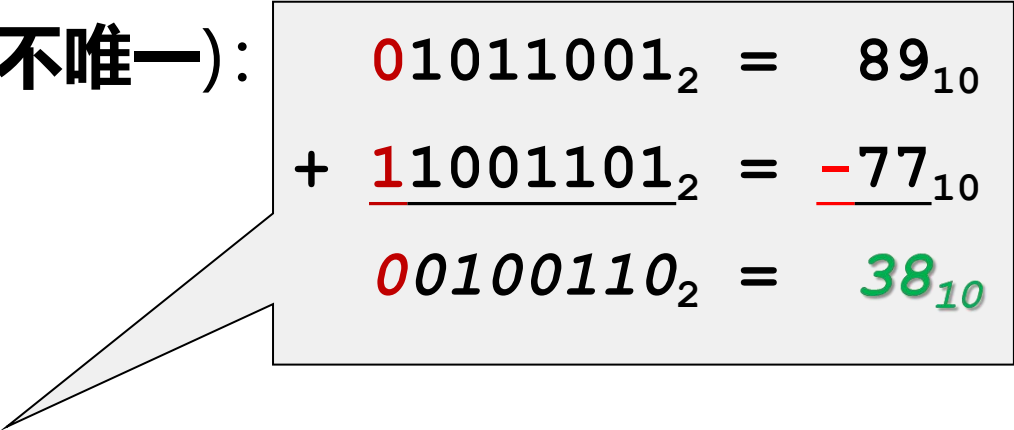
- $[+0]_{\text{原}} = 0\ 000\dots 0$

- $[-0]_{\text{原}} = 1\ 000\dots 0$

- 加、减运算方式不统一

- 符号相异加法，不能直接用原码运算（使减法不能转换成加法来运算）。

- 因此从50年代开始，整数都采用**补码**来表示。



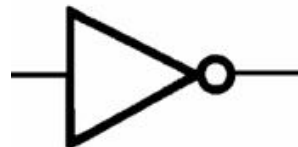
01011001₂ = 89₁₀
+ 11001101₂ = -77₁₀
00100110₂ = 38₁₀

2. 反码表示法

- **正数的反码**: 与原码相同;
- **负数的反码**: 符号位也是1, 数值位是将原码的数值位 按位取反 .
- 反码的符号位表示方法 与原码相同 .
- 例, 反码表示: $7_{10} = 00111_2$
 $-7_{10} = 11000_2$
- $[0.1111]_{\text{反}} = 0.1111$ $[-0.1111]_{\text{反}} = 1.0000$
- $[+1110]_{\text{反}} = 01110$ $[-1110]_{\text{反}} = 10001$
- $[+0]_{\text{反}} = 0000\cdots0$ $[-0]_{\text{反}} = 1111\cdots1$.

反码特性

- 正零负零两个零表示 (不同)
- 取反用**逻辑非门**容易实现
- 与原码类似: 减法运算实现仍很复杂。





3. 补码 表示法

- 正数的补码: 符号位为0; 正数补码与其原码、反码相同;
- 负数的补码: (单独) 符号位为1; 数值部分逐位取反, 再末位加1.

$$\begin{aligned} & \text{■ } [-1010\ 1010]_{\text{补}} \\ & = 1\ 0101\ 0101 \\ & \quad + \quad \quad \quad \underline{\quad\quad\quad 1} \\ & = 1\ 0101\ 0110 \end{aligned}$$

$$\begin{aligned} & \text{■ } [-0.010101]_{\text{补}} \\ & \text{■ } = 1.101010 \\ & \quad + \quad \quad \quad \underline{\quad\quad\quad 1} \\ & \text{■ } = 1.101011 \end{aligned}$$

例子

$$\begin{aligned} \blacksquare X &= -0.11111111 \\ [X]_{\text{补}} &= 1.00000000 \\ &+ 0.00000001 \\ &= 1.00000001 \end{aligned}$$

$$\begin{aligned} \blacksquare X &= -0.00000000 \\ [X]_{\text{补}} &= ??? \\ [X]_{\text{补}} &= \textcolor{teal}{1}.1111\ 1111 \\ &\quad + \underline{0.0000\ 0001} \\ &= \underline{\textcolor{blue}{1}0}.0000\ 0000 \text{ (舍掉符号位进位)} \\ &= \textcolor{red}{0.00000000}。 \end{aligned}$$

补码特性

- 零的补码形式唯一：

$$[+0.0000]_{\text{补}} = [-0.0000]_{\text{补}} = 0.0000$$

- 补码相对 原码(反码) 少一个负零。

双符号位补码（变形补码）

$$\text{整数 } [X]_{\text{补}} = \begin{cases} X & 0 \leq X < 2^n \\ 2^{n+2} + X & -2^n \leq X < 0 \end{cases}$$

例：正数 **00** 01010110 负数 **11** 01010001

$$\text{小数 } [X]_{\text{补}} = \begin{cases} X & 0 \leq X < 1 \\ 4 + X & -1 \leq X < 0 \end{cases}$$

例： $x = -0.1001$

变形补码 $[x]_{\text{补}} = \mathbf{11.0111}$ 。

求 $x = -0.101$ 的**变形**补码：

解： $[x]_{\text{反}} = (\text{[填空1]})$

变形补码 $[x]_{\text{补}} = (\text{[填空2]})$ 。

补码的加减法

由补码定义得：

$$[X + Y]_{\text{补}} = [X]_{\text{补}} + [Y]_{\text{补}}$$

$$[X - Y]_{\text{补}} = [X]_{\text{补}} + [-Y]_{\text{补}}$$

$$[-Y]_{\text{补}} = [[Y]_{\text{补}}]_{\text{补}}$$

求补：对 $[Y]_{\text{补}}$ 逐位取反，再末位加1。

补码加减法运算实例

$x=0.1011$, $y=-0.0101$ 用变形补码(双符号位补码) 求 $x+y$, $x-y$.

解: $[x]_{\text{补}} = 00.1011$, $[y]_{\text{补}} = 11.1011$

$[-y]_{\text{补}} = \underline{00.0101}$ 先确定符号位更好

$x+y$

$$\begin{array}{r} 00.1011 \\ + 11.1011 \\ \hline (\text{舍}) \underline{1} 00.0110 \end{array}$$

$x-y$

$$\begin{array}{r} 00.1011 \\ + 00.0101 \\ \hline (\text{溢出}) \underline{01}.0000 \end{array}$$

补码特性 (补充)

- 唯一的零
- 符号位可以直接参与运算 (针对定点数, 即纯整数/纯小数)
- 减法可以变成加法, 使加、减运算的电路统一。

4、移码表示法

■ 移码仅用于表示定点整数(纯整数), 见谭教材P23

■ $[x]_{\text{移}} = x + 2^n$, n 为 x 数值部分的二进制位数

□ 例1: $x = +1011111$

原码为 0 1011111

反码为 0 1011111

补码为 0 1011111

移码为 11011111 。

4、移码表示法

例2: $x = -1011111$

原码为 $1\ 1011111$

反码为 $1\ 0100000$

补码为 $1\ 0100001$

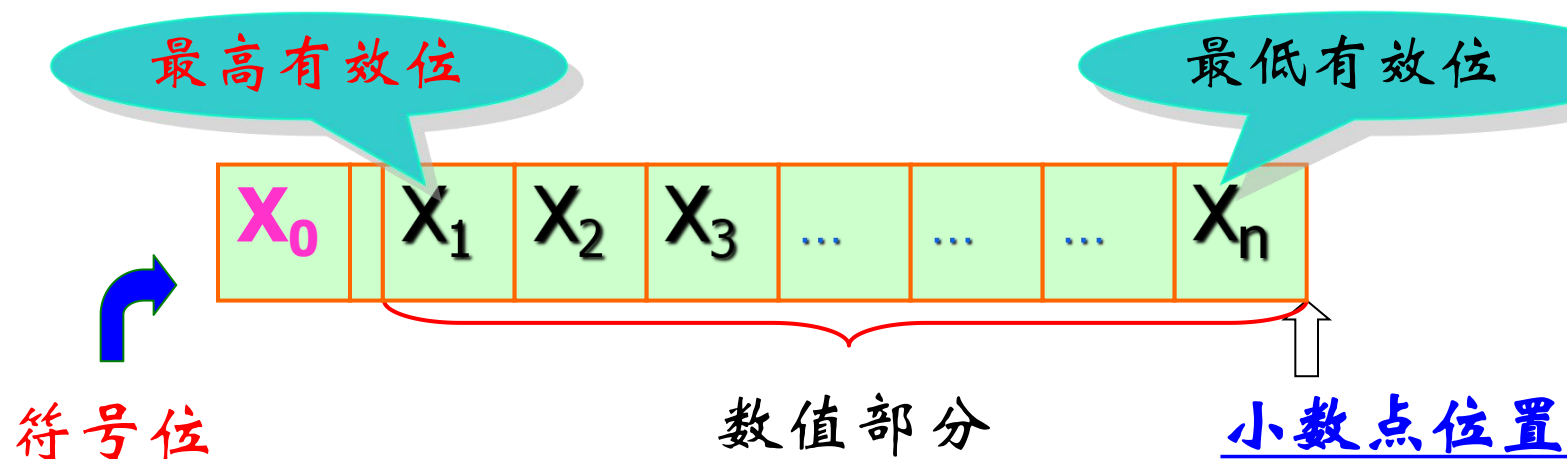
移码为 $0\ 0100001$

结论: 移码和补码数值位相同, 符号位相反。

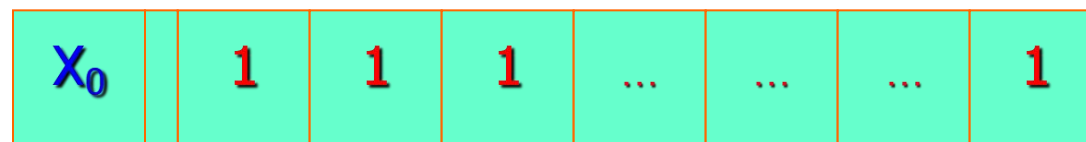
数的定点、浮点表示方法 (✖)

主要内容:

- 定点数表示 (小数点位置固定的数)
 - 定点整数 (纯整数)
 - 定点小数 (纯小数)
 - 定点数仅能表示纯小数、纯整数
- 浮点数表示 (解决小数点问题) 。

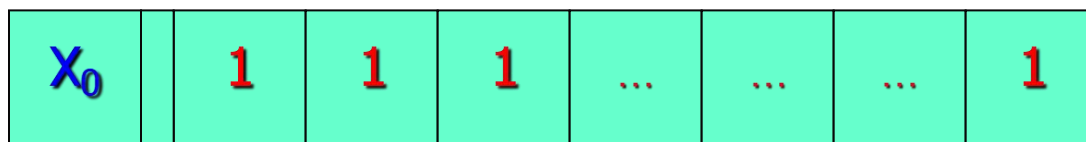
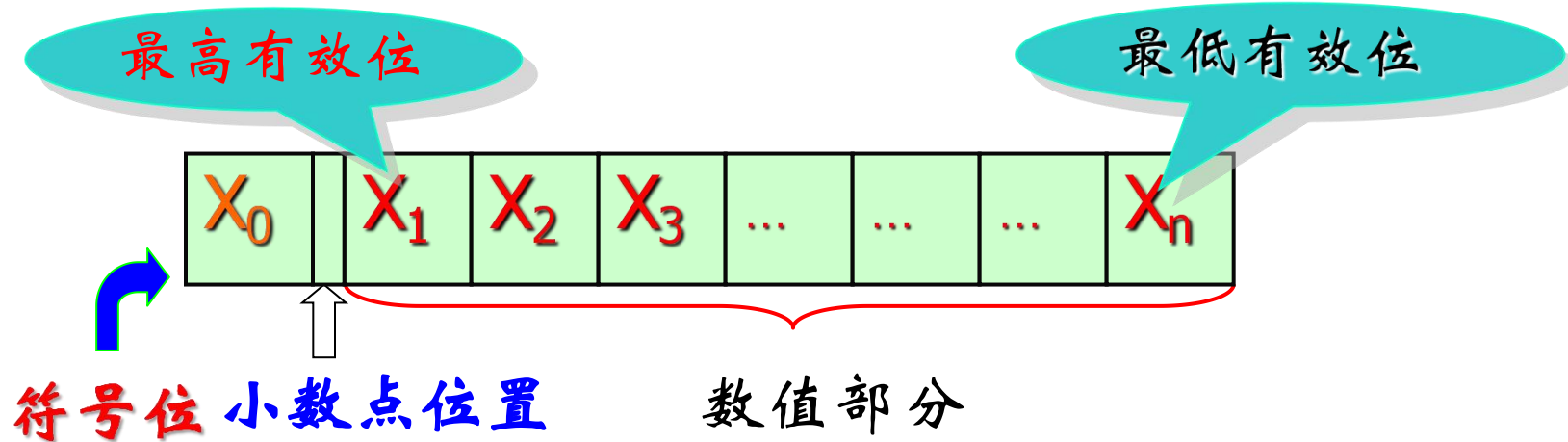


若数值位采用原码：



$$0 \leq |X| \leq 2^n - 1$$

定点小数：表示纯小数



$$0 \leq |X| \leq 1 - 2^{-n}$$

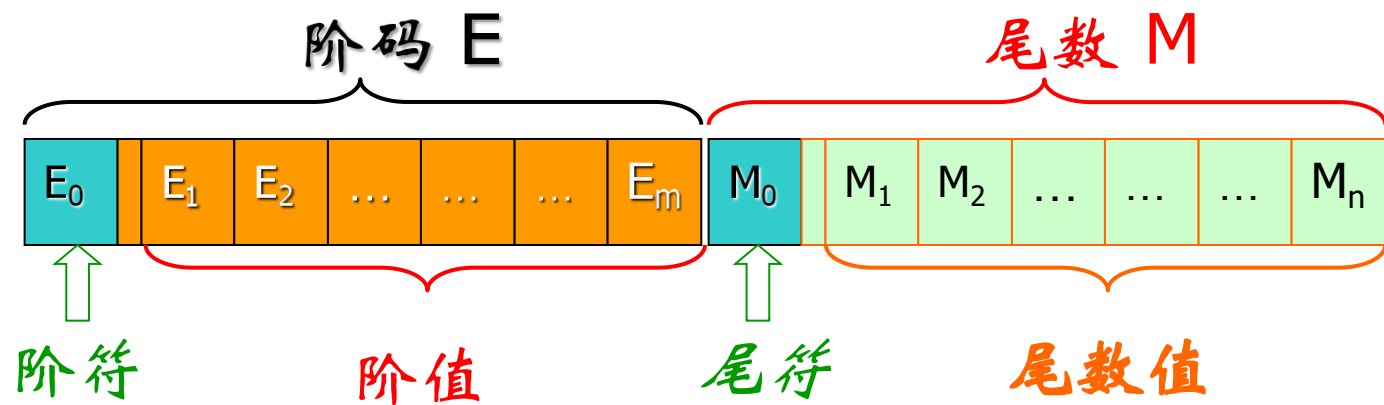


浮点数如何表示? (重+难)

- 参与运算的数据通常既包括整数也包括小数部分 (无法直接用定点数表示); 如何表示?
- 唯一性问题: 浮点数的指数表达, 小数点位置其实不固定; 同样是 0.5 :
 - 十进制 0.05×10^1 50×10^{-2} 5×10^{-1}
 - 二进制 $0.1B \times 2^0$ $1B \times 2^{-1}$ $0.01B \times 2^1$
- 因此——科学记数法:
 - 让数据表示唯一 → “规格化” :
 - 十进制数的表示 $N = M \times 10^e$, $1 \leq |M| < 10$ (尾数)
 - 二进制数 $N = M \times 2^e$, $1 \leq |M| < 2 \rightarrow$ IEEE754标准。

浮点数的表示

$$\begin{aligned} N &= 2^E \times M \\ &= 2^{\pm e} \times (\pm m) \end{aligned}$$



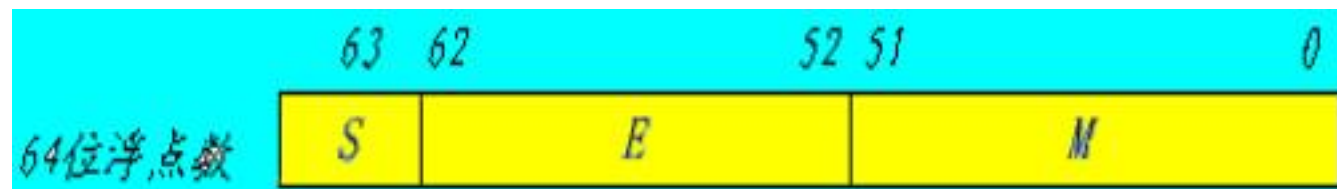
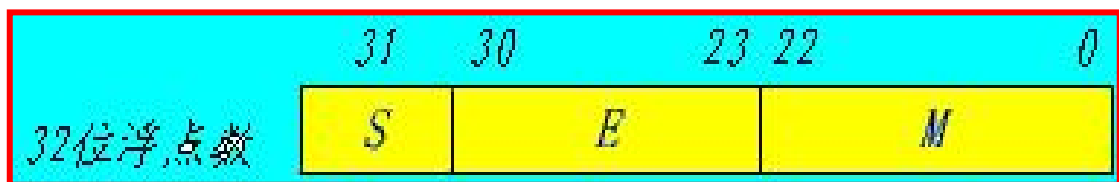
浮点数的规格化问题

- 尾数的**绝对值**最高有效位**是1**的数称为 规格化数 .
- 绝对值 而言, 2种规格化数: **0.1XXXXX** 1.XXXXXX (IEEE754)
- **正的** 8位 **定点**小数 可表示的范围:
 - 0.0000001 --- 0.1111111B
- 设 8位**浮点数**中, 阶码的阶值3位, 尾数的尾数值3位 (**规格化**)
 - 可表示**浮点数**的绝对值 $2^{-111\text{B}} * \mathbf{1}.000\text{B}$ --- $2^{111\text{B}} * \mathbf{1}.111\text{B}$
 - 即 0.0000001B --- 11110000B
- 以上比较可知: 浮点数的表示范围 比定点数的大、精度高。

浮点数标准 IEEE754 (重+难)

IEEE委员会发布的 IEEE754标准 规定:

- **32位单精度** 和64位双精度 浮点数的基本格式
- 规定**尾数M用原码, 阶码E用移码表示。**



数据格式

□ 32位的浮点数:

- ◆ S是该数的符号位, 1位, 在最高位, “0”表示正数, “1”表示负数。
- ◆ M 尾数域(绝对值), 23位, 在编码的低位部分, 采用定点纯小数表示。
- ◆ E是阶码, 8位, 采用移码表示
 - 尾数域M最左位(最高有效位)总是1, 故这一位通常不予存储, 而认为隐藏在小数点的左边。
 - 采用这种方式时, 将浮点数的指数 的真值e 变成 阶码E (共8bit) 时, 应将指数e加上一个固定的偏移值 127 (01111111 B), 即:

$$E = e + 127$$

- 因此, 规格化的32位浮点数x 的真值为

$$x = (-1)^S \times (1.M) \times 2^{E-127}$$

$$\text{指数} e = E - 127。$$



浮点数转换实例

例1：若浮点数x 的754标准存储格式为 $(41360000)_{16}$ ，求x对应的十进制数值。

解：将16进制数展开后，可得二进制数格式为

0 100 0001 0 011 0110 0000 0000 0000 0000

符号S 阶码E(8位) 尾数M(23位)

指数 $e = E - 127 = 10000010B - 127 = 130 - 127 = 3$

包括隐藏位1的尾数：

$1.M = 1.011\ 011\ 0\ 0000\ 0000\ 0000\ 0000 = 1.011011$

于是有

$$x = (-1)^S \times 1.M \times 2^e$$

$$= +(1.011011\ B) \times 2^3 = +1011.011B = (11.375)_{10}。$$

浮点数转换实例

例2: 将数 20.59375转换成754标准的32位浮点数的二进制存储格式 (写成十六进制形式)。

解: 首先分别将整数和小数部分转换成二进制数:

$$20.59375 = 10100.10011 \text{ B}$$

然后移动小数点, 使小数点在第1, 2位之间

$$10100.10011 = \underline{1.0100 \ 10011} \text{ B} \times 2^4 \quad (\text{规格化, } 1.M)$$

$\therefore e=4$ 于是得到:

$$S=0, E=4+127=131, M=0100 \ 10011$$

最后得到32位浮点数的二进制存储格式为:

$$0100 \ 0001 \ 1010 \ 0100 \ 1100 \ 0000 \ 0000 \ 0000 = (41A4C000)_{16}。$$

注意:

- **十进制浮点数并不一定能精确表示成二进制浮点数, IEEE754标准中的尾数域M有时只是近似数, 与实际值存在微小误差, 但失之毫厘, 谬以千里, 在实际工程设计中需要警惕!**
- **海湾战争中美国的“爱国者”导弹由于这样的浮点误差, 直接导致飞毛腿导弹拦截失败, 美军军营多人死伤——小小浮点带来的是致命灾难和重大损失! 大家需注意严谨的科学工匠精神, 需提升工程素养!**

- Binary Coded Decimal 二进制编码的十进制
- 有几种BCD码, 其中最常用:
 - 8421码 $(8*X_3+4*X_2+2*X_1+1*X_0)$ 有权码
 - 余三码 $(8*X_3+4*X_2+2*X_1+1*X_0) + 0011$
- 详见P31表2.8。

BCD码运算问题

- 8421码的校正

- $8 + 7 = 1000 + 0111 = 1111$ 非法编码

- 余三码的校正

- $0 + 0 = 0011 + 0011 = 0110 = 3$ 非法编码

- $4 + 4 = 0111 + 0111 = 1110 = ?$ 非法编码

- 相对而言运算比较复杂。



2.4 数据信息的校验

这部分只能放到下个模块(逻辑代数基础)中讲

- 解决编码在时间、空间上传输可靠性问题
- 编码中引入一定冗余，增加编码的最小码距，使编码符合某种规则，当编码出现一个错误时就成为非法代码
 - 奇偶信息的校验
 - 海明校验
 - CRC循环冗余校验。



奇偶校验

■ 奇校验:

- 使整个校验码（数据 + 校验位）中1的个数为奇数
- 数据 0000 → 0000 1 （奇校验）
- 0001 → 0001 1 （偶校验）

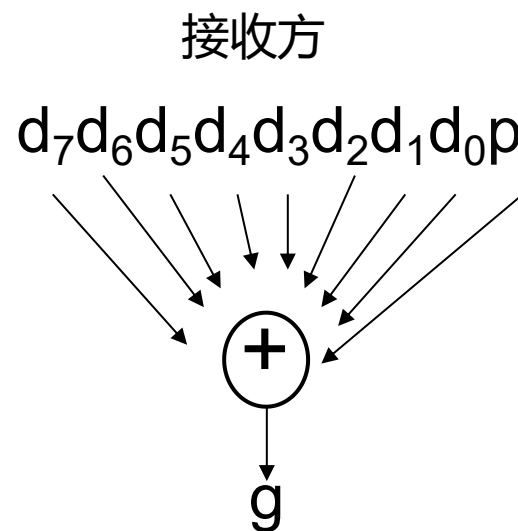
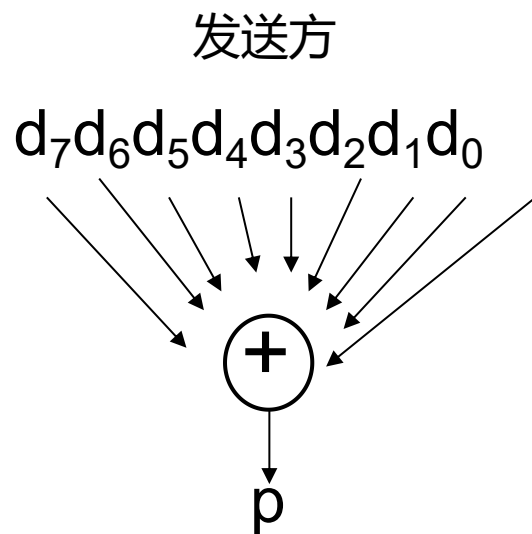
偶校验: $P = D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8$

奇校验: $P = \overline{D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8}$

偶校验
检错码: $G = P \oplus D_1 \oplus D_2 \oplus D_3 \oplus D_4 \oplus D_5 \oplus D_6 \oplus D_7 \oplus D_8$

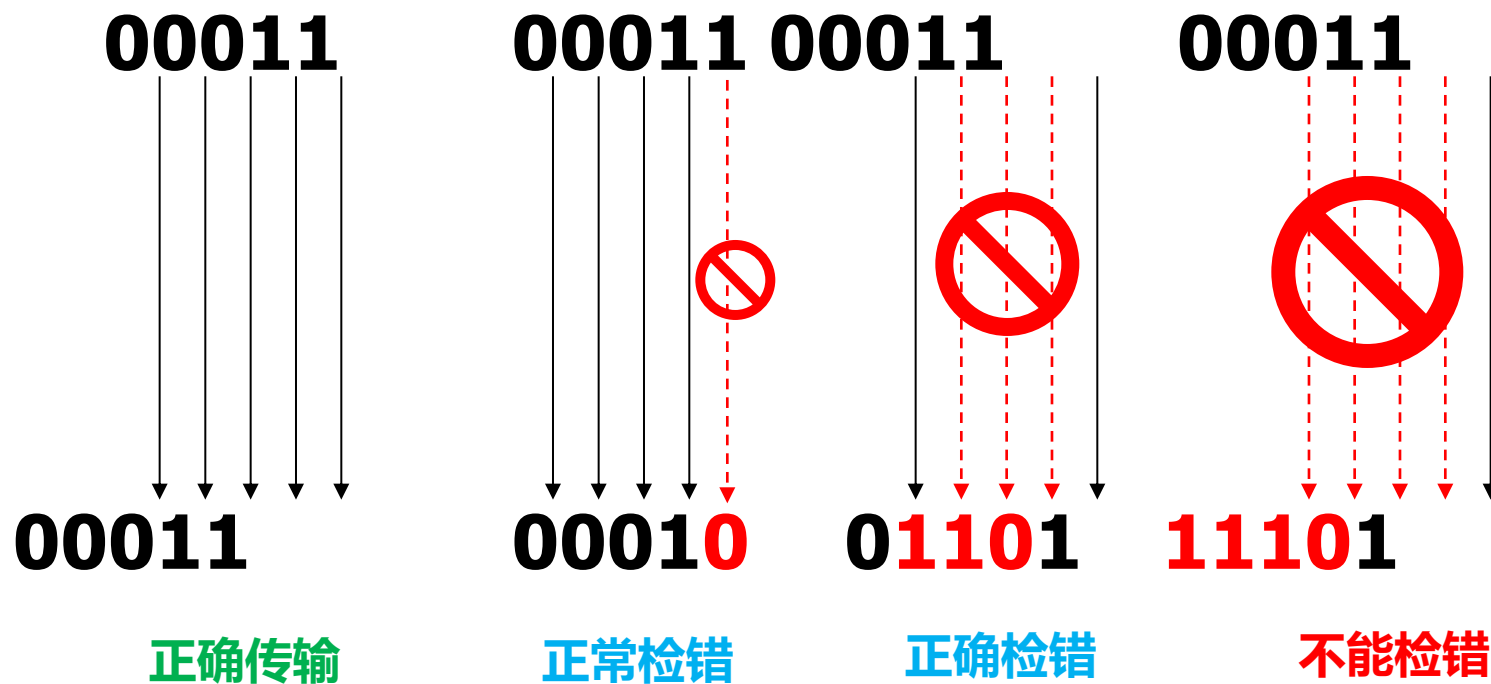
G=0 表示数据正常，否则表示出错。

奇偶校验



- 发送方生成校验码=数据+校验位
- 接收方利用校验码生成检错码g
- 检错码不为零表示错误发生
- **检错码为零时, 是否表示数据一定正确?**

偶校验



缺：仅能识别奇数个错误，不能纠正错误，也不能保证正确

优：实现简单，编码效率高。

作业 2

- 本子上完成：
- 1. 试题书P13--计算题: 7.
- 2. 试题书P17: 22 . 25. 参考课上例题过程, 试题书解析中写的过程不一定好.
- 3、教材P55习题 2.9 (1) 最后结果写成十六进制形式、 2.10题 .
- 本周会在QQ群上, 收第2章共2次主观题作业, 请大家做好准备!

The End

作业讲解

- 1. 试题书P11--2.3计算题 2. : 应该是由数据字长、原码规则 和定点表示(例如定点整数) 的规则, 来写出原码表示的最大正数(例如最大正整数) 的编码, 再相应计算出它的值。原码表示的最小负整数的求法 类似。

不同机器码公式对比

	$-2^{\underline{n}} \leq x \leq 0$	$-1 \leq x \leq 0$	$x \geq 0$
原码	$2^n - x$	$1 - x$	x
反码	$2^{n+1} - 1 + x$	$2 - 2^{-n} + x$	x
补码	$2^{n+1} + x$	$2 + x$	x
移码	$2^n + x$	无	$2^n + x$

机器码小结

- 用符号位表示数符
- 4种定点编码方式
 - 原码 用来表示浮点（实）数的尾数
 - 反码 已不用于表示数值数据
 - 补码 50年代开始成为整数标准
 - 移码 用于浮点数的阶码
- 补码优势
 - 模运算，加、减运算统一
 - 唯一0，方便使用。

参考前面例子, 将以下十六进制数转换为二进制数。
写出计算过程:

$$\begin{aligned} & (A4.8)_{16} \\ &= (1010 \text{ [填空1] } . \text{ [填空2] })_2 \\ &= (\text{ [填空3] })_2 \quad \text{注意结果的化简。} \end{aligned}$$